

DuocUC[®] 

Herencia

DSY1102 - Desarrollo Orientado a Objetos



¿Qué aprenderemos hoy?

01

¿Qué es la herencia?

02

¿Por qué es útil?

03

Superclase y Subclase

04

Sintaxis básica con extends

05

Ejemplo con clase Animal

06

Subclases Perro y Gato

07

Sobreescritura de métodos.

08

Uso de super y constructores

09

Jerarquía de clases (Animales)

10

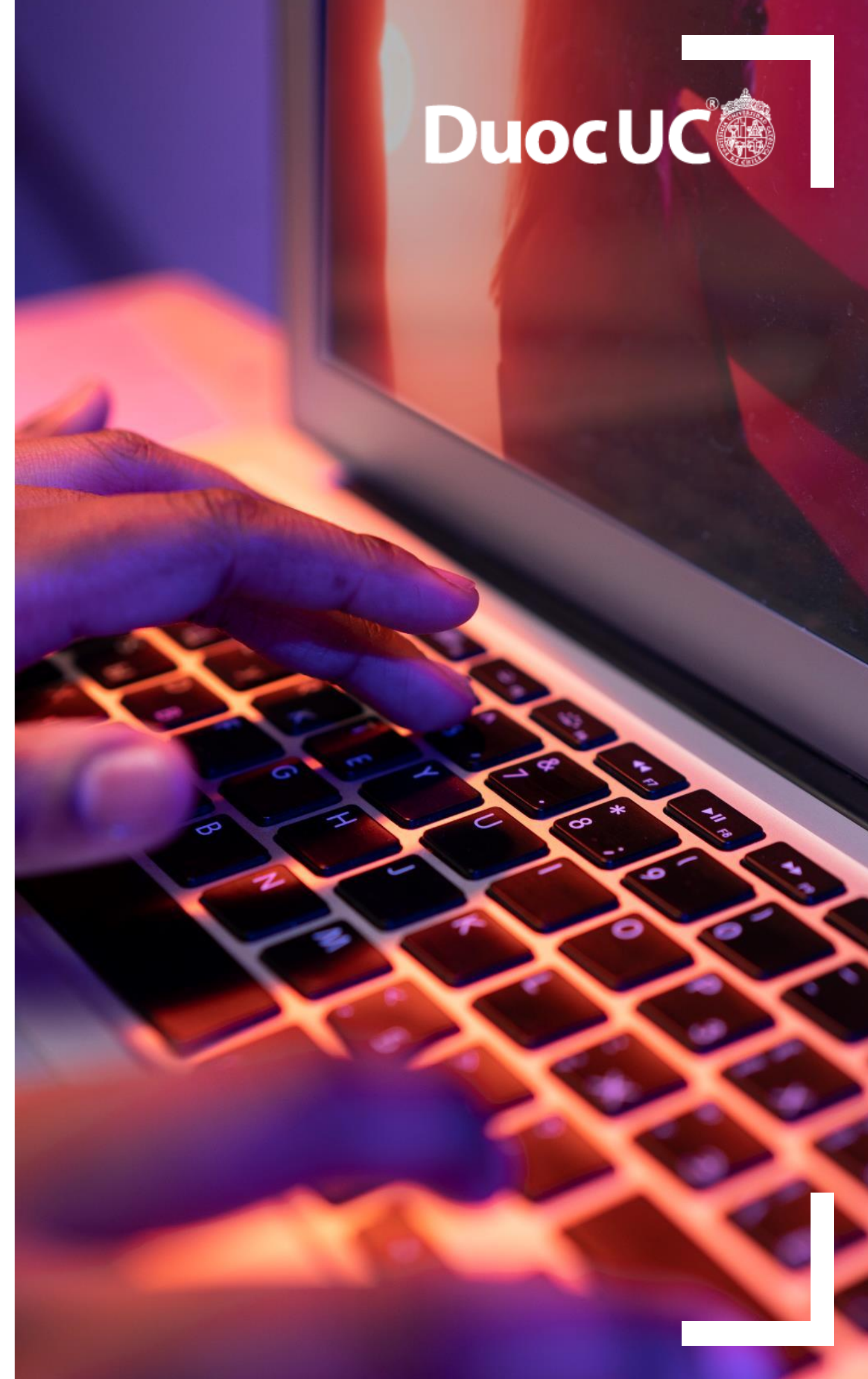
Herencia y polimorfismo básico.

11

Buenas practicas

12

Checklist de herencia



</> Programación Orientada a Objetos

Java

Herencia en Java

Reutilización, especialización y polimorfismo con ejemplos de animales.



Reutilización
Evita código duplicado



Especialización
Añade nuevo comportamiento

```
Perro.java

public class Animal {

    public void hacerSonido () {
        System.out.println("...");
    }

    public class Perro extends Animal {

        @Override
        public void hacerSonido() {
            System.out.println("Guau!");
        }

    }

}
```



¿Qué es la Herencia?

Reutilización de código entre clases relacionadas

Definición y Claves

Es un pilar de la POO donde una **subclase** adquiere automáticamente los atributos y métodos de una **superclase** existente.



Reutilización

Escribes el comportamiento común una sola vez en la clase base.



Especialización

Las subclases pueden añadir características únicas o modificar las heredadas.



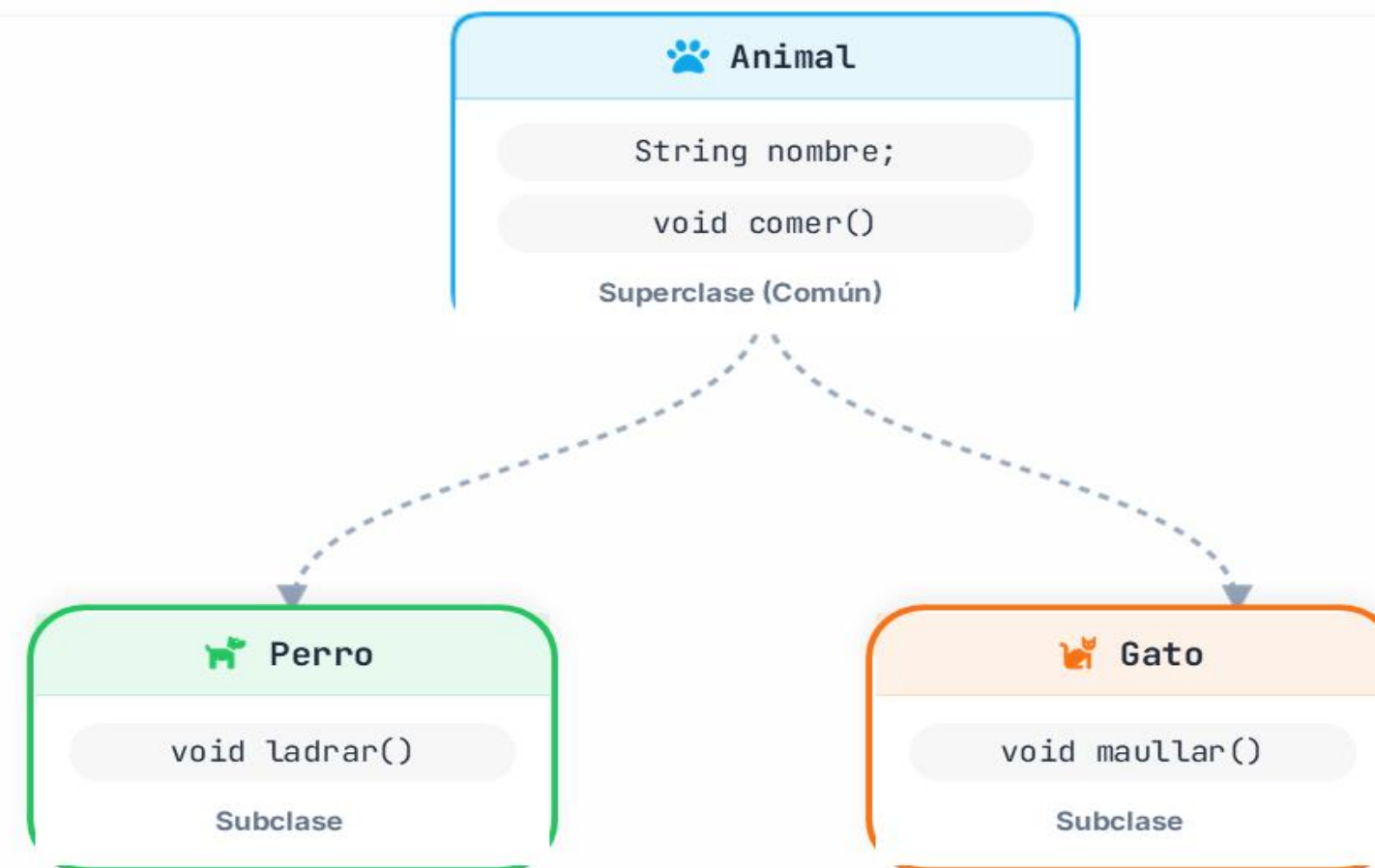
Relación "Es un"

Establece una jerarquía natural (ej. un Perro es un Animal).

Concepto Base:

✓ Herencia = Superclase → Subclases

Estructura Jerárquica: Animales



↓ Hereda atributos y métodos

+ Añade comportamiento específico





¿Por qué es útil la herencia?

Reutilización de código y organización jerárquica

★ Beneficios Principales

La herencia permite crear nuevas clases basadas en clases existentes, obteniendo sus características automáticamente.



Reutilización de código

Aprovecha métodos y atributos ya definidos en la superclase.



Menos duplicación

Centraliza el comportamiento común evitando copiar y pegar código.



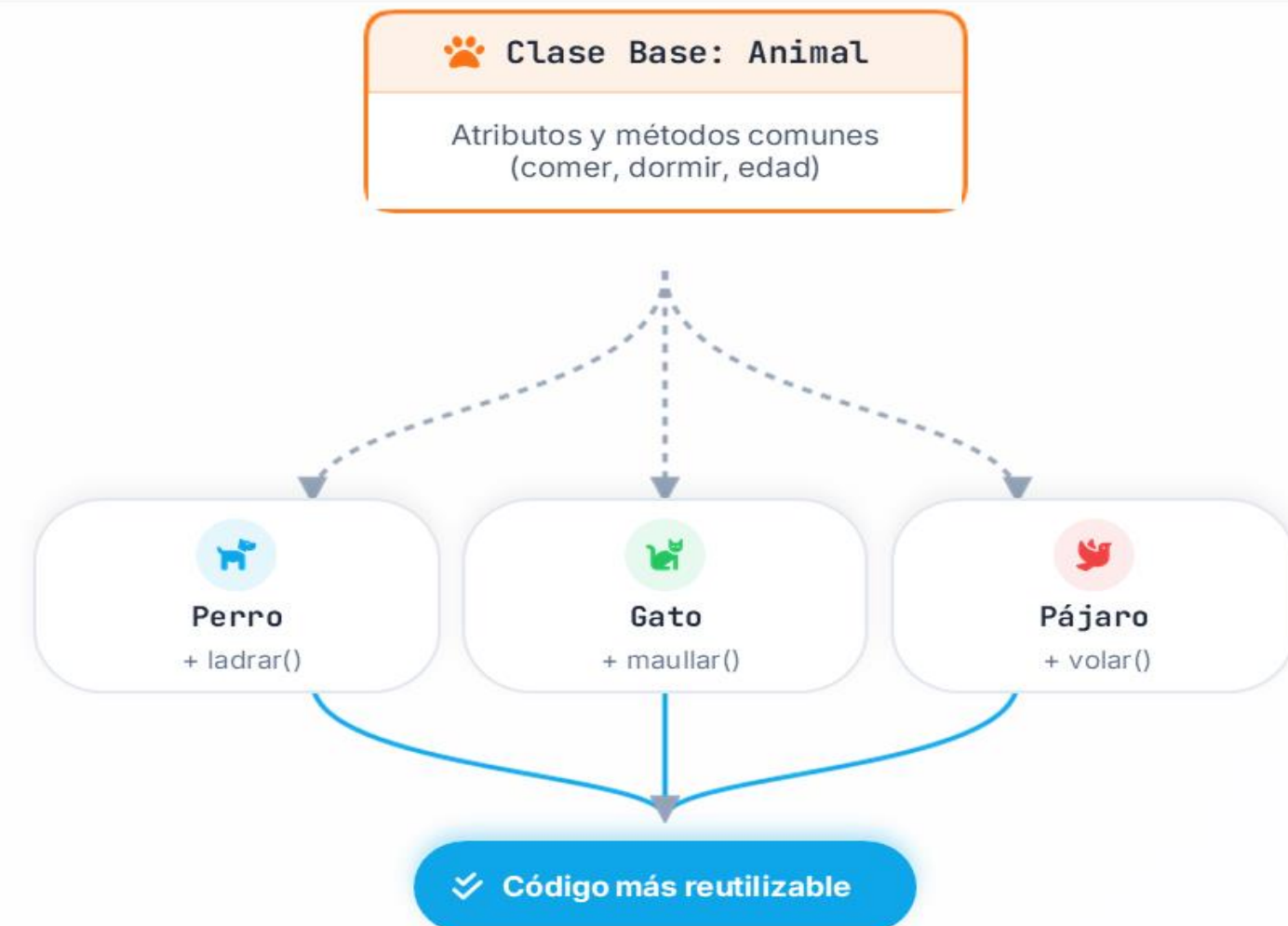
Organización jerárquica

Modela relaciones del mundo real ("es-un") de forma clara y lógica.

Resultado Final:

✓ Código más mantenible y escalable

Flujo de Herencia



Estructura Lógica



Menos Duplicación





Superclase y Subclase

La base de la jerarquía en Java



Conceptos Clave

La herencia establece una relación jerárquica entre clases, donde unas derivan de otras para reutilizar y extender comportamiento.



Superclase (Animal)

Proporciona atributos y métodos comunes. Actúa como plantilla genérica.



Subclase (Perro)

Hereda de Animal. Añade comportamientos específicos como ladrar.



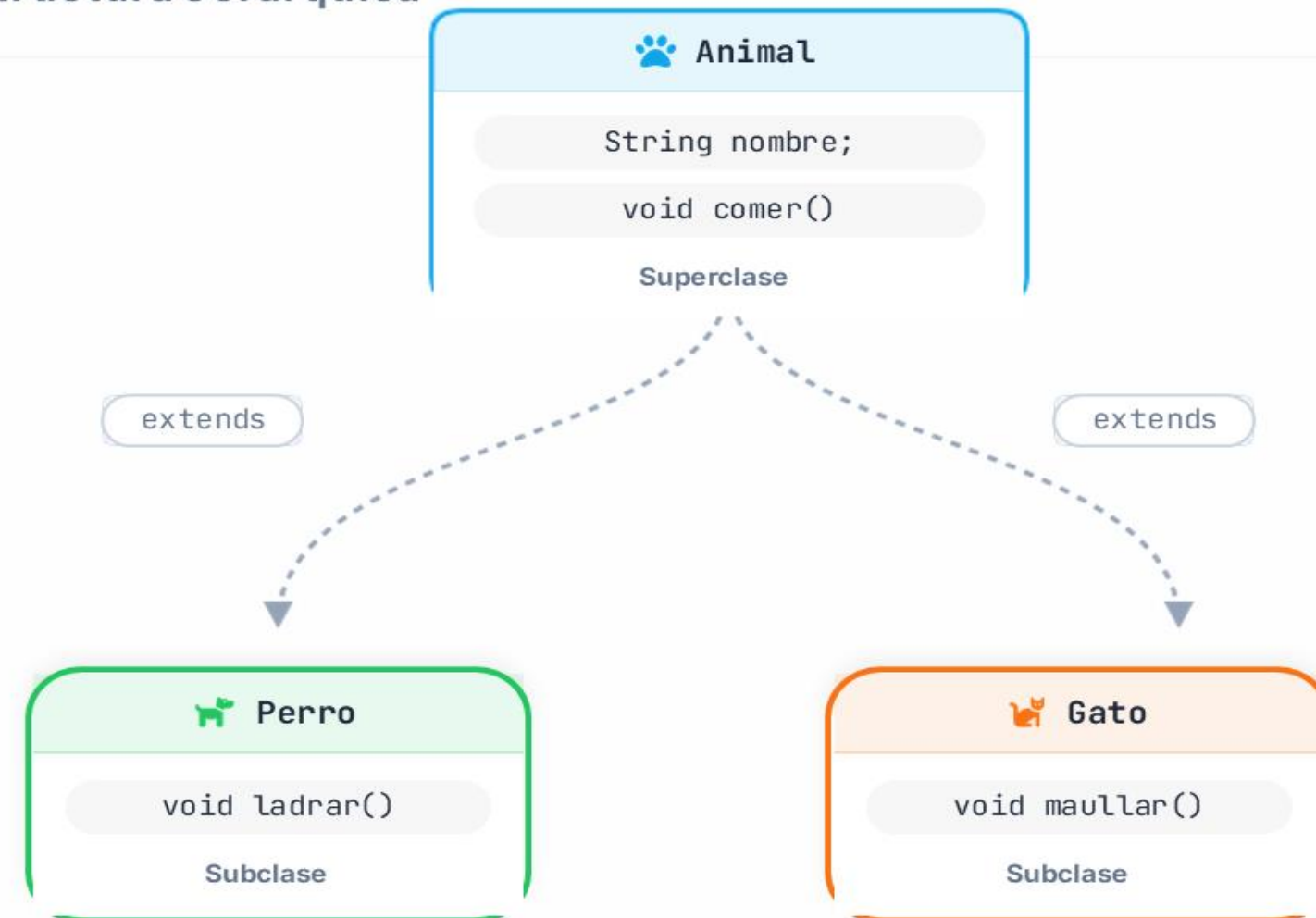
Subclase (Gato)

Hereda de Animal. Añade comportamientos como maullar.

Relación "Es-un":

Un Perro **es un** Animal. Todo lo que hace Animal, lo hace el Perro (y más).

Estructura Jerárquica



Base General



Especialización



</> Sintaxis Básica con `extends`

Declarando relaciones de herencia en Java

3 Pasos para Heredar

Implementar herencia en Java es directo. Solo necesitas la palabra clave **extends** para conectar clases.



1. Define la Superclase

Crea la clase base con los atributos y métodos comunes.



2. Usa `extends`

Declara la subclase usando `extends NombreSuperclase`.



3. Usa lo Heredado

Instancia la subclase y llama a métodos de la superclase.

Regla de Oro:

 En Java solo se puede heredar de una clase a la vez.

Flujo de Implementación



1. Superclase

```
class Animal { ... }
```

extends



2. Subclase

```
class Perro extends Animal { }
```

hereda



3. Uso de Métodos

```
miPerro.comer(); // De Animal
```


Ejemplo base: clase Animal

La superclase centraliza características comunes

La Base del Diseño

La clase **Animal** actúa como una plantilla general que agrupa los atributos y comportamientos que comparten todas las especies, evitando escribir el mismo código varias veces.



Estado Común (Atributos)

Todo animal tiene un nombre y una edad.



Acciones Básicas (Métodos)

Acciones universales como `hacerSonido()` y `dormir()`.

Punto Clave:

Escribe una vez, hereda muchas

Estructura de la Superclase

`class Animal`

Atributos (Estado)

A `String` `nombre` # `int` `edad`

Métodos (Comportamiento)

 `hacerSonido()`
"..."

 `dormir()`
"Zzz..."



Subclases: Perro y Gato

Especializando clases base con comportamiento propio

Especialización

Las subclases **heredan** los atributos y métodos de la superclase, y pueden añadir sus propias características específicas.



Reutilización

No necesitamos reescribir nombre, edad o métodos como comer() y dormir().



Extensión

Perro añade el método ladrar() y Gato añade el método maullar().



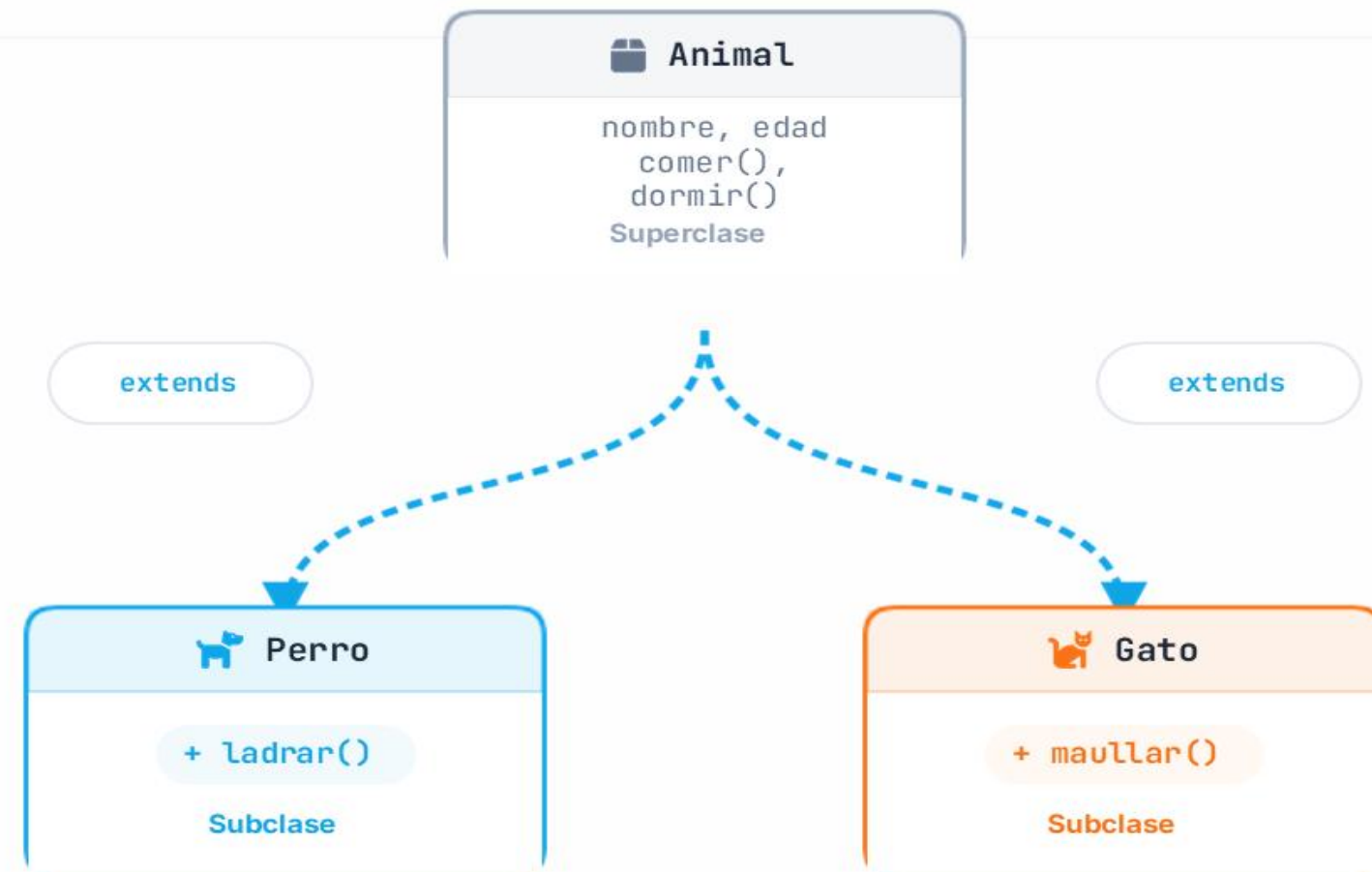
Relación "Es un"

Un Perro "es un" Animal. Un Gato "es un" Animal.

Resultado Final:

 Código modular y especializado

Jerarquía de Clases

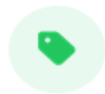


Sobrescritura de métodos

Redefiniendo comportamiento en subclases

Redefinir Comportamiento

Las subclases proporcionan una implementación específica para un método ya definido en su superclase, adaptándolo a sus necesidades.



@Override

Anotación que valida la sobrescritura y previene errores tipográficos.



Misma Firma

El método debe tener idéntico nombre, tipo de retorno y parámetros.



Polimorfismo

Base del comportamiento dinámico en tiempo de ejecución.

Beneficio Clave:

- ✓ Especialización segura de código

Flujo de Sobrescritura

Clase Animal

```
public void hacerSonido() {  
    // Sonido genérico  
}
```

@Override

Clase Perro

```
public void hacerSonido() {  
    print("¡Guau!");  
}
```

Resultado: "¡Guau!"

@Override

Clase Gato

```
public void hacerSonido() {  
    print("¡Miau!");  
}
```

Resultado: "¡Miau!"



↑ Uso de super y constructores

Llamando a la clase padre en la herencia

⚡ El poder de super

La palabra clave **super** actúa como un puente hacia la superclase, permitiendo reutilizar su inicialización.



Invocar Constructores

`super (...)` llama al constructor de la clase padre.



Acceder a Métodos

`super.metodo ()` ejecuta la versión original sobrescrita.



Regla Estricta

Debe ser la primera instrucción en el constructor.

Resultado:

✓ Inicialización en cadena segura

Flujo de Inicialización



Animal(nombre, edad)

Inicializa los atributos comunes

`super(nombre,
edad);`



Perro(nombre, edad, raza)

Añade atributos específicos

Inicialización en Cadena: Al crear un Perro, primero se invoca obligatoriamente a `super ()` para que *Animal* establezca el estado base.



Jerarquía de Animales

Organizando tipos relacionados mediante niveles de abstracción

Organización Estructural

Una jerarquía organiza tipos relacionados, permitiendo clasificar entidades desde lo más general (superclase) hasta lo más específico (subclases).



Nivel Base (General)

La clase Animal agrupa características y comportamientos universales.



Nivel Intermedio

Categorías amplias como Mamífero o Ave que heredan de Animal.



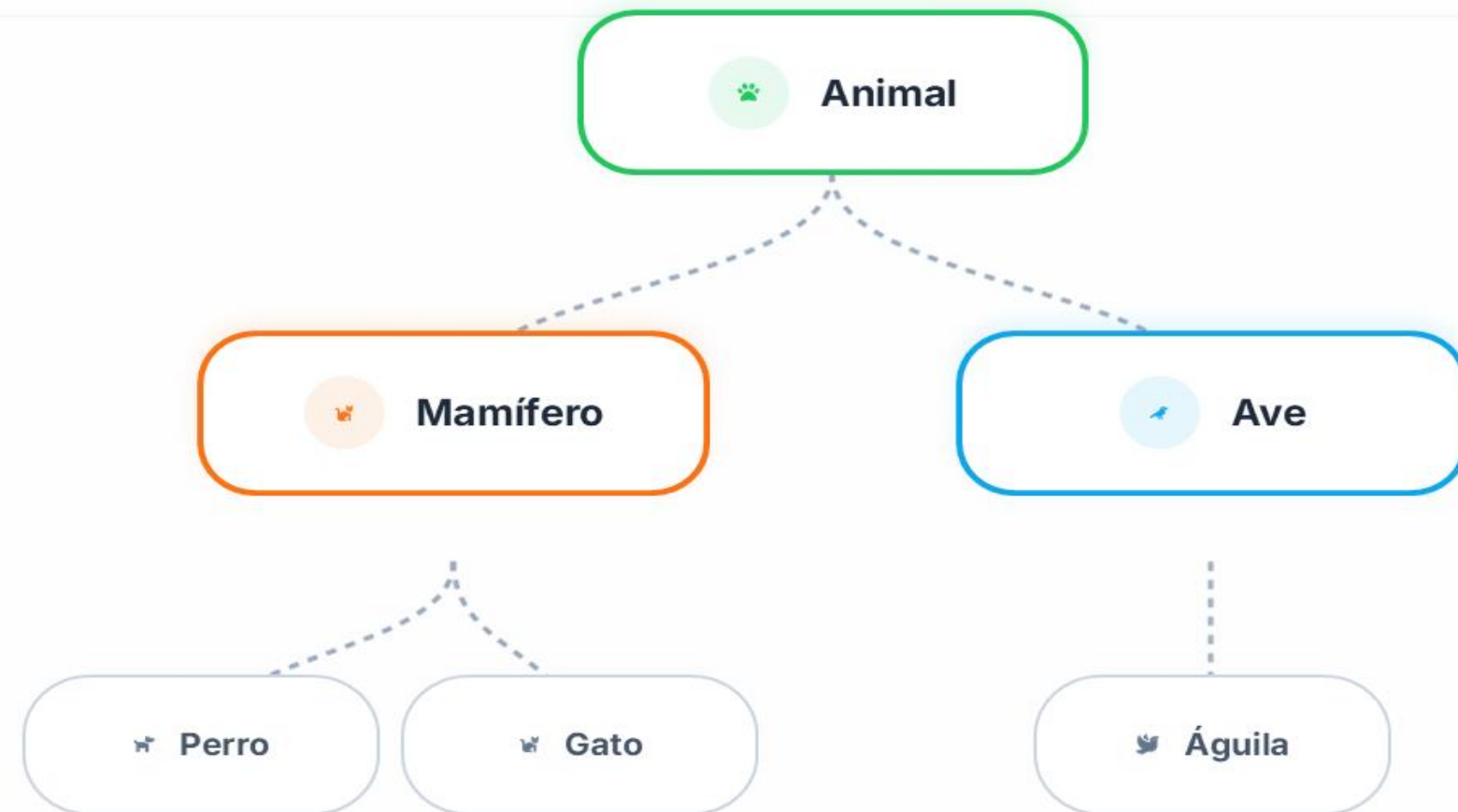
Nivel Específico

Clases concretas como Perro, Gato o Águila con rasgos únicos.

Resultado Final:

🔗 Código estructurado y lógico

Árbol de Herencia Simple



Herencia y Polimorfismo Básico

Una misma referencia, múltiples comportamientos

Polimorfismo

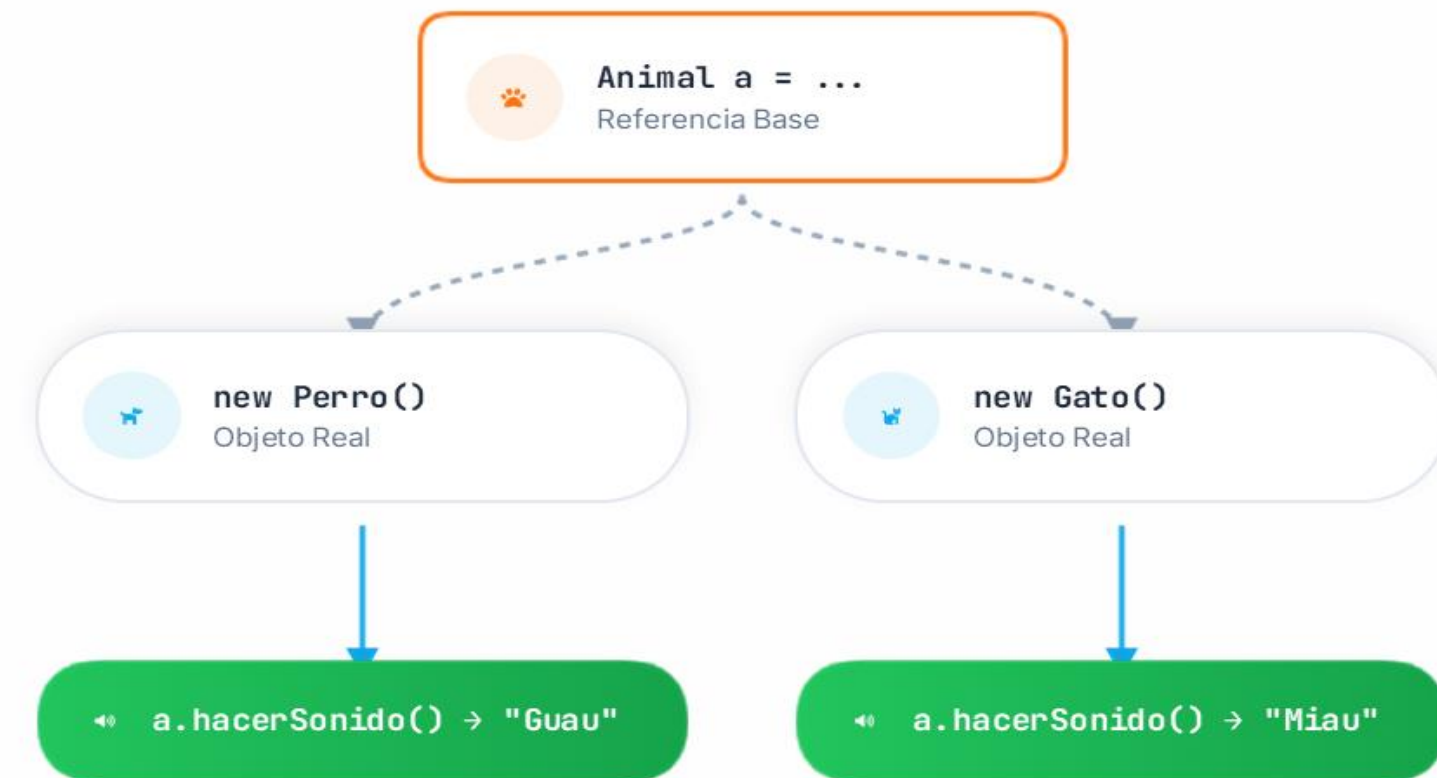
Gracias a la herencia, una referencia del tipo **superclase** (como `Animal`) puede apuntar a un objeto de cualquier **subclase** (como `Perro` o `Gato`).

Al llamar a un método sobrescrito, se ejecutará la versión correspondiente al **objeto real** en memoria.

Ventajas Clave:

- ✓ Código más genérico
- ✓ Fácil de extender
- ✓ Interfaz unificada

Flujo Polimórfico



Buenas Prácticas al Usar Herencia

Reglas de oro para un diseño orientado a objetos sólido

✓ Reglas Fundamentales

Aplicar la herencia de manera disciplinada transforma un conjunto de clases en una arquitectura robusta.



Relación "Es un"

Hereda solo si la subclase es un subtipo verdadero (ej. Perro es un Animal).



Evitar jerarquías profundas

Mantén un árbol plano. Jerarquías largas son frágiles y difíciles de entender.



Sobrescritura con claridad

Utiliza la anotación `@Override` siempre para evitar errores al redefinir.



Reutilizar lo común

Concentra atributos y lógica compartida en la superclase para no duplicar código.

Beneficio Clave:

♥ **Arquitectura Limpia y Mantenible**

Mapa de Buenas Prácticas





Errores comunes y checklist

Puntos clave para diseñar jerarquías robustas

⚠ Antipatrones a Evitar

Evita estos errores comunes al aplicar herencia para no crear sistemas frágiles y acoplados.



Herencia vs Composición

Usar herencia ("es un") cuando corresponde composición ("tiene un").



Jerarquías Profundas

Crear demasiados niveles de herencia dificulta el mantenimiento.



Duplicar Lógica

No aprovechar métodos de la superclase y reescribir lógica.



No usar @Override

Omitir la anotación al sobrescribir, ocultando errores de compilación.

Resultado:

✔ Diseño Resiliente

Checklist de Verificación



Consejo: Aplica este checklist antes de usar 'extends' en tus proyectos Java.





Conclusiones y Próximos Pasos

Cierre de Herencia en Java

✓ Síntesis Final

La herencia permite **reutilizar código**, organizar objetos jerárquicamente y crear diseños más limpios y escalables en Java.



Reutilizar

Evita duplicación con la clase base.



Especializar

Añade características únicas en subclases.



Sobrescribir

Modifica el comportamiento con @Override.

Resultado:

🌱 Código Mantenable y Polimórfico

Ecosistema de Herencia



Clase Base Animal

Atributos comunes



Subclases (Perro)

Especialización



Comportamiento

Polimórfico en ejecución

→ Próximos Pasos Recomendados

1

Practicar con más animales

2

Composición vs Herencia

3

Interfaces en Java



Felicitaciones ¡Has finalizado el módulo!

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO

 Comisión Nacional
de Acreditación
CNA-Chile

NIVEL DE EXCELENCIA
HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la
calidad / Vinculación con el Medio / Investigación, creación y/o innovación